

Lie Theory, Featuring 3D Rotations

Brian Beckman

Mar 2026

In this notebook, we formalize the mathematical framework of Lie theory, using 3D rotations as a driving example. While we have previously applied Lie theory to rotational dynamics, we took many mathematical facts on faith. Here, in a structured, cheat-sheet style, we derive everything, proceeding from the central concept, the $\mathfrak{su}(2)$ Lie algebra, to the $SO(3)$ and $SU(2)$ rotational Lie groups, which emerge naturally via *representation theory*.

In short, this notebook serves as a physics-centric introduction to Lie theory, showing techniques that generalize far beyond 3D rotations.

Preliminaries

Lie Ontology

One of the most important applications of Lie theory is for 3D rotations. In two earlier notebooks, *SO(3) for Classical Mechanics* and *SU(2) for Classical Mechanics*, we showed how to solve dynamical problems via geodesics in the Lie groups $SO(3)$ and $SU(2)$, with metric tensors composed of kinetic energies. We demonstrated the superiority of $SU(2)$ in avoiding the gimbal lock in the Euler-angle parameterization of $SO(3)$, by honoring the intrinsic, simply connected topology of the configuration space of rotations. In those notebooks, we developed just enough of the mathematical theory to assure the application, in a “just-in-time” style. This style left us with several pieces of theory “on the table.” Here, we organize those pieces and fit them together in a tidy structure that’s easy to understand, to remember, and to be creative with. We first give an inventory of the objects and mappings in the theory, and then present a diagram linking them together. In the rest of the notebook, we work out the details formally, machine-checked by Wolfram.

Object Inventory

The “big picture” for rotations is that we can **represent** the abstract Lie algebra $\mathfrak{g}$ of $\mathfrak{su}(2)$ in at least two ways. One of them leads us to $SO(3)$ and its troubles with gimbal lock (via the non-trivial *fundamental group* $\pi_1(SO(3)) = \mathbb{Z}_2$), the other to $SU(2)$ with no such troubles and, thus, a virtuous slippery slope to quantum theory.

- *Representation* is a formal notion: *mapping the abstraction to matrices*. Representation theory itself is an immense topic. Here we need just a basic understanding.

- Abstractly, a Lie algebra is a vector space with a **Lie bracket operation**, $[A, B]$, that satisfies the Jacobi identity, $[A, [B, C]] + [C, [A, B]] + [B, [C, A]] = 0$. In a matrix representation, the Lie bracket is the matrix commutator $[A, B] = A \cdot B - B \cdot A$.

The abstract algebra, $\mathfrak{g}$, for 3D rotations is characterized by its specific structure constants. In general, the **structure constants** f_{ijk} of any Lie algebra-*qua*-vector-space are the Lie bracket applied to a choice of basis vectors: $[E_i, E_j] = f_{ijk} E_k$ (sum over k). The specific structure constants for rotations are ϵ_{ijk} , the fully antisymmetric Levi-Civita tensor, where i, j, k are in $\{1, 2, 3\}$. These structure constants model the cross product, familiar from elementary physics as the primary tool for describing rotation. In the real domain, these structure constants imply the generators J_k . In the complex domain, they imply the generators T_k . The corresponding Lie groups follow via the exponential map.

- A Lie group is both a group and a differentiable manifold. As a manifold, it enjoys parameterization by coordinate charts that must be differentiable where they overlap. As a group, its elements describe linear transformations of other objects, spinors and mat-vecs in our running example.

- The adjoint action, $Q \cdot v \cdot Q^\dagger$ is linear in v .

The double-covering of $SO(3)$ by $SU(2)$ follows from the topology of their respective manifolds. $SO(3)$ is diffeomorphic to the real projective space $\mathbb{RP}^3$, visualized as the 3-sphere S^3 , where every point is identified with its antipode, $(q \sim (-q))$. $\mathbb{RP}^3$ is not simply connected because any closed loop in S^3 that includes both q and $-q$, the same point in $\mathbb{RP}^3$, is a closed loop in $\mathbb{RP}^3$ that cannot be shrunk to zero size without “tearing”. In contrast, $SU(2)$ is diffeomorphic to S^3 . This space is simply connected because any closed path can be shrunk to zero size.

- S^3 is the set of all points $q \equiv (q_0, q_1, q_2, q_3)$ in $\mathbb{R}^4$ such that $(q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1)$.
- In $\mathbb{RP}^3$, points are actually pairs $(q, -q)$ of elements of $\mathbb{R}^4$.

The elements of each group describe linear transformations of geometric objects that are not elements of the group. Concretely, we *represent* the linear transformations as $n \times n$ matrices that operate on n -vectors. We represent $SO(3)$ by real 3×3 rotation matrices that act on real 3-vectors. We represent $SU(2)$ by complex 2×2 matrices, isomorphic to *unit quaternions*, that act on spinors in $V_{\text{spin}} = \mathbb{C}^2$ and on complex, 2×2 , traceless, Hermitian matrices (**mat-vecs** in $\mathfrak{h}_\sigma$) that represent real 3-vectors.

- The algebra $\mathfrak{su}(2)$ is isomorphic to the pure imaginary quaternions, $(0, q_1, q_2, q_3)$.

Finally, we introduce some notation, ad_X and Ad_Q , for actions in the Lie algebra and the Lie Group respectively, and the Killing form for extracting components of members of the Lie groups.

Out[4]=

Short Name	Item Description	Context / Physical Interpretation
$\mathfrak{g}$	Abstraction of $\mathfrak{su}(2)$	Abstract Lie algebra: $[E_i, E_j] = \epsilon_{ijk} E_k$ (no sum)
f_{ijk}	Structure Constants	Numbers ϵ_{ijk} characterizing the bracket
σ_k	Pauli Matrices	Basis $\{\sigma_1, \sigma_2, \sigma_3\}$ for the Pauli Map
T_k	Generators of $\mathfrak{su}(2)_{\mathbb{C}^2}$	$T_k = -\frac{i}{2} \sigma_k$; $[T_i, T_j] = T_k$ in $\mathbb{C}^2 \times \mathbb{C}^2$
J_k	Generators of $\mathfrak{su}(2)_{\mathbb{R}^3}$	$[J_i, J_j] = J_k$ in $\mathbb{R}^3 \times \mathbb{R}^3$
$\mathfrak{su}(2)_{\mathbb{C}^2}$	$\mathfrak{su}(2)$ 2x2 $\mathbb{C}$ Rep	Complex matrix representation (Generators T_k)
$\mathfrak{su}(2)_{\mathbb{R}^3} = \mathfrak{so}(3)$	$\mathfrak{so}(3)$ 3x3 $\mathbb{R}$ Rep	Real matrix representation (Generators J_k)
$SU(2)$	$SU(2)$ Group	2 x 2 unitary matrices, $\det=1$
$SO(3)$	$SO(3)$ Group	3 x 3 orthogonal matrices, $\det=1$
V_{spin}	Spinors ($\mathbb{C}^2$)	Space for Fundamental Action of $SU(2)$
$\mathbb{R}^3$	Real $\mathbb{R}^3$	Euclidean space of physical 3-vectors
$\mathfrak{h}_\sigma$	Mat-Vecs	Traceless Hermitian 2 x 2 matrices
$\mathcal{Z}$	Center	The kernel $\{I, -I\}$ of the $SU(2) \rightarrow SO(3)$ map
S^3	Topology S^3	Topology of $SU(2)$ (3-sphere)
$\mathbb{R}^3P$	Topolog $\mathbb{R}^3P$	Topology of $SO(3)$ (Projective space)
$\text{ad}_X Y$	Infinitesimal adjoint	Algebra action $[X, Y]$
$\text{Ad}_Q Z$	Finite Adjoint	Group action $Q \cdot Z \cdot Q^\dagger$
B	Killing Form	Trace inner product for coordinate extraction

Mapping Inventory

Here we list, non-exhaustively, various mappings amongst the objects above. **MatrixExp** takes us from a representation of an element of a Lie algebra to a representation of an element of a Lie Group. This is a surjection in that multiple inputs can produce the same output. Not listed is the inverse, **MatrixLog**, which goes the other way, because it is multi-valued, but there is an example in the body of the notebook of how to choose a branch of that inverse.

- Technically, **MatrixExp** is always a surjection exactly when the target Lie group is compact and connected. $SO(3)$ and $SU(2)$ satisfies these criteria. See Hall, Knapp, and Milnor in the References.

The **center** $\mathcal{Z}$ of a group is the set of all elements that commute with every other element of the group. For $SU(2)$, the center consists of only I , the identity, and $-I$, the negative identity. The **inclusion map** f merely states that $f(I) = I$ and $f(-I) = -I$. This inclusion formally ensures that $SU(2)/\mathcal{Z}$ is isomorphic to $SO(3)$ in that both I and $-I$ in $SU(2)$ map to I in $SO(3)$, expressing the “double cover” relationship between $SU(2)$ and $SO(3)$. In group-theoretic terms, $\mathcal{Z}$ is the **kernel** of the $SU(2) \rightarrow SO(3)$ map, the set of elements in the domain that map to the identity element.

The **Pauli map** takes 3-vectors to traceless, Hermitian matrices (mat-vecs) in $\mathfrak{h}_\sigma$. The trace projection takes them back, via a circuitous derivation involving the Killing Form, B . Examples are shown in the body of the notebook.

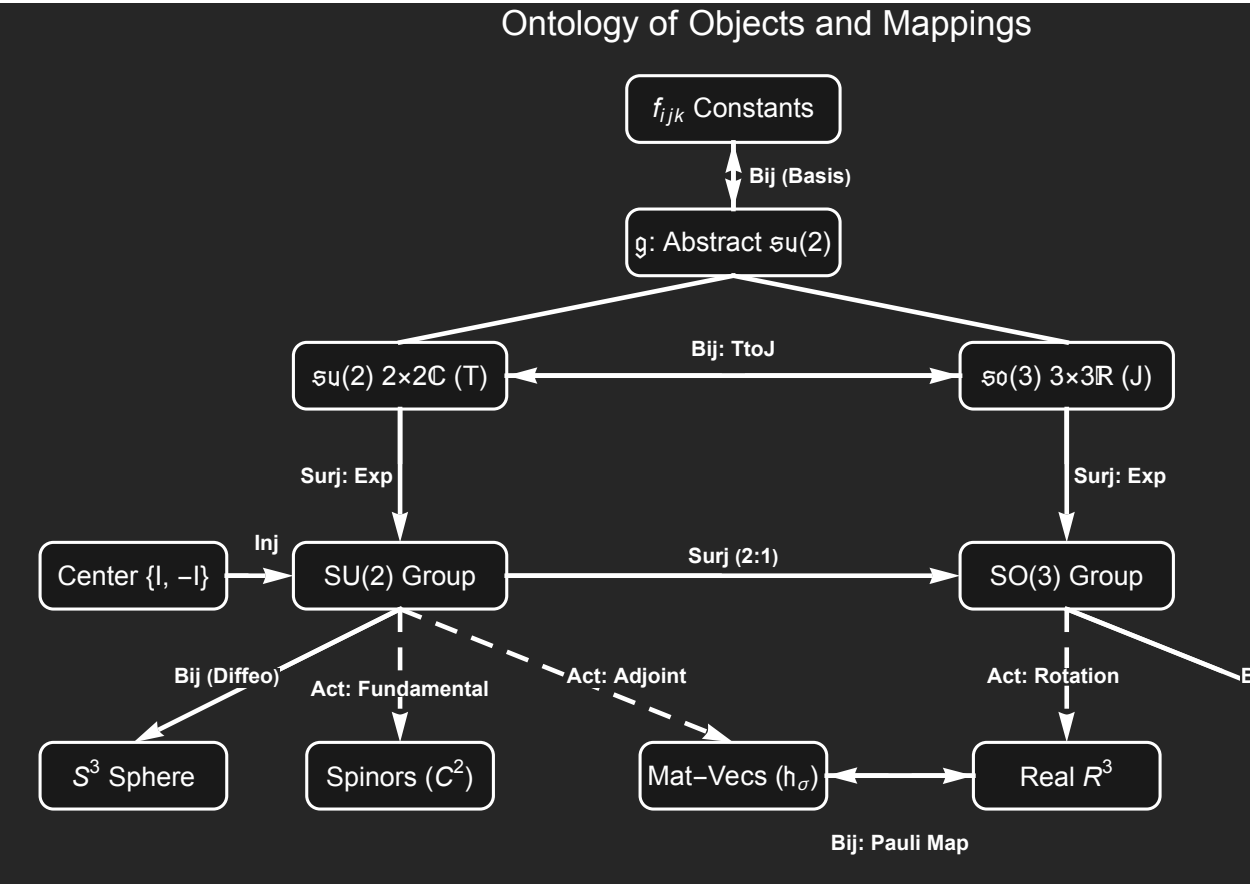
To concretely represent the vectors in the Lie-algebra-*qua*-vector space, we must choose a basis. Many choices map to the same structure constants f_{ijk} , and that’s why the mapping is only surjective. However, once a basis is chosen for $\mathfrak{su}(2)$ and the structure constants are materialized as the specific constants ϵ_{ijk} , the mapping becomes bijective in that we can recover the basis from the particular structure constants ϵ_{ijk} .

The remaining rows of the Mapping Inventory should be self-explanatory given the previous explanations.

Source	Target	Name / Mechanism	Type
$\mathfrak{su}(2)$	$SU(2)$	MatrixExp	Surj
$\mathfrak{so}(3)$	$SO(3)$	MatrixExp	Surj
$SU(2)$	$SO(3)$	Adjoint Rep / 2-to-1 Cover	Surj (2:1)
$\mathbb{Z}$	$SU(2)$	Center inclusion	Inj
$\mathbb{R}^3$	$\mathfrak{h}_\sigma$	Pauli Map ($\mathbf{v} \cdot \boldsymbol{\sigma}$)	Bij
$\mathfrak{h}_\sigma$	$\mathbb{R}^3$	Trace Projection (getTCoeffs)	Bij
$\mathfrak{su}(2)$	$\mathfrak{so}(3)$	Algebra Ad Rep (TtoJ)	Bij
$\mathfrak{g}$	f_{ijk}	Basis Mapping	Surj
$SU(2)$	S^3	Topo ID	Bij (Diffeo)
$SO(3)$	$\mathbb{R}^3P$	Topo ID	Bij (Diffeo)
$SU(2) \times V_{\text{spin}}$	V_{spin}	Fundamental Action ($Q \psi$)	Act
$SU(2) \times \mathfrak{h}_\sigma$	$\mathfrak{h}_\sigma$	Adjoint Action ($Q Z Q^\dagger$)	Act
$SO(3) \times \mathbb{R}^3$	$\mathbb{R}^3$	Standard Rot ($R \mathbf{v}$)	Act

Ontology Diagram

This diagram clarifies the relationships between the objects and mappings. These relationships will be clarified in the proofs in the body of the notebook.



Adjoint Representation

$\mathfrak{so}(3)$ from Real Generators

Derived from the structure constants ϵ_{ijk} of the fundamental representation.

```
In[7]:= ClearAll[J, comm, commTest, triples, 0];
In[8]:= Echo[MatrixForm /@
  (J = { {0 0 0}, {0 0 -1}, {0 1 0} }, { {0 0 1}, {0 0 0}, {-1 0 0} }, { {0 -1 0}, {1 0 0}, {0 0 0} } ),
  "so(3) J basis of su(2) :"];
» so(3) J basis of su(2): { {0 0 0}, {0 0 -1}, {0 1 0} }, { {0 0 1}, {0 0 0}, {-1 0 0} }, { {0 -1 0}, {1 0 0}, {0 0 0} }
```

Cyclic Commutation of J

Demonstration that $[J_i, J_j] = J_k$ for all even permutations $(i, j, k) = (1, 2, 3)$

```
In[9]:= ClearAll[comm, commTest, triples];
comm[M_][i_, j_] := M[[i]].M[[j]] - M[[j]].M[[i]];
commTest[M_][i_, j_, k_] := comm[M][i, j] == M[[k]];
triples = Echo[NestList[RotateLeft, Range[3], 2], "index triples :"];
proof[And@@commTest[J]@@@triples];
» index triples : {{1, 2, 3}, {2, 3, 1}, {3, 1, 2}}
QED ■
```

Cyclicity From Levi-Civita

```
In[14]:= ClearAll[ε];
ε = LeviCivitaTensor[3];
ClearAll[cyclicity];
cyclicity[M_] :=
  And@@Flatten@Table[
    comm[J][i, j] ==
      Sum[J[[k]] * ε[[i, j, k]], {k, 3}],
    {i, 3}, {j, 3}];
proof@cyclicity[J];
QED ■
```

Basis of $SO(3)$ from $\mathfrak{so}(3)$

A convincing example of one elementary rotation matrix from one generator.

```
In[19]:= Echo[MatrixForm /@ (MatrixExp[θ #] & /@ J), "eθ J ="];
» eθ J = { {1, 0, 0}, {0, Cos[θ], -Sin[θ]}, {0, Sin[θ], Cos[θ]} }, { {Cos[θ], 0, Sin[θ]}, {0, 1, 0}, {-Sin[θ], 0, Cos[θ]} }, { {Cos[θ], -Sin[θ], 0}, {Sin[θ], Cos[θ], 0}, {0, 0, 1} }
```

Round-Tripping $SO(3) \longleftrightarrow \mathfrak{so}(3)$

```
In[20]:= ClearAll[roundTripLieGroupAndAlgebra];
roundTripLieGroupAndAlgebra[M_] :=
  And @@ Flatten@ (FullSimplify[
    MatrixLog[MatrixExp[θ M[[#]]]],
    Assumptions → -π < θ < π] == θ M[[#]] & /@ Range[3]);
proof@roundTripLieGroupAndAlgebra[J];
QED ■
```

Fundamental Representation

$\mathfrak{su}(2)$ from Complex Generators

```
In[23]:= ClearAll[T]
Echo[MatrixForm /@
  {T = -i/2 PauliMatrix /@ {1, 2, 3}},
  "su(2) T basis :"];
proof[And @@ commTest[T] @@@ triples];
» su(2) T basis : { {0, -i/2}, {-i/2, 0}, {0, i/2}, {i/2, 0}, {0, 0}, {0, 0} }
```

QED ■

Cyclic Commutation of T

```
In[26]:= proof@cyclicality[T];
QED ■
```

$SU(2)$ from $e^{\mathfrak{su}(2)}$

```
In[27]:= Echo[MatrixForm /@ (MatrixExp[θ #] & /@ T), "eθ T ="];
» eθ T = { { Cos[ $\frac{\theta}{2}$ ]   -i Sin[ $\frac{\theta}{2}$ ] }, { Cos[ $\frac{\theta}{2}$ ]   -Sin[ $\frac{\theta}{2}$ ] }, { e- $\frac{i\theta}{2}$    0 },
            { -i Sin[ $\frac{\theta}{2}$ ]  Cos[ $\frac{\theta}{2}$ ] }, { Sin[ $\frac{\theta}{2}$ ]   Cos[ $\frac{\theta}{2}$ ] }, { 0   e $\frac{i\theta}{2}$  } }
```

Round-Tripping $SU(2) \longleftrightarrow \mathfrak{su}(2)$

```
In[28]:= proof@roundTripLieGroupAndAlgebra[T];
```

QED ■

$\mathfrak{su}(2) \approx$ Pure Imaginary Quaternions

Members of $\mathfrak{su}(2)$ are the 2×2 complex, traceless, anti-Hermitian matrices. They correspond to pure-imaginary quaternions.

```
In[29]:= Module[{su2 = { { a + b i   c + d i },
                        { -c + d i  -a - b i } }},
  proof[Echo[Tr[su2] == 0, "su2 matrices are traceless :"]];
  proof[{a → 0} == Echo[Solve[su2 == -su2* // ComplexExpand, {a}][[1],
    "su2 matrices are anti-Hermitian iff a == 0 :"]];]
```

» su2 matrices are traceless : True

QED ■

» su2 matrices are anti-Hermitian iff a == 0 : {a → 0}

QED ■

Topologies

$SU(2)$ Has Topology of S^3

We will prove this in two steps. The first step is to show that the unitary requirement and the unit-determinant requirement for members of $SU(2)$ imply that the matrix representation has a specific form, namely

$$\text{form} = \begin{pmatrix} a + bi & c + di \\ -c + di & a - bi \end{pmatrix}$$

where $\{a, b, c, d\}$ are all real. During the course of that demonstration, we will find (and exploit) that $a^2 + b^2 + c^2 + d^2 = 1$.

In the second step, we show that the determinant of the special form is exactly $a^2 + b^2 + c^2 + d^2$, which we already know is 1. Interpreting this as the expression for the radius of a sphere in $\mathbb{R}^4$ with coordi-

nates $\{a, b, c, d\}$, we have the final result.

```
In[30]:= Module[{
  (* arbitrary 8-parameter complex matrix *)
  M =  $\begin{pmatrix} a + b i & c + d i \\ e + f i & g + h i \end{pmatrix}$ ,
  vars = {a, b, c, d, e, f, g, h},
  complexEqns, realEqns, linearEqns, soln},
  (* complex equations *)
  Echo[(complexEqns = Flatten[{
    (* unitarity *)
    Thread[Flatten[M.M† // ComplexExpand] == Flatten[IdentityMatrix[2]]],
    (* det = 1 ("special" property *)
    Det[M] == 1}]) // ColumnForm, "complex eqns :"];
  (* Split into real equations *)
  Echo[(realEqns = Select[#, # != True &] @ Flatten[
    Table[{
      ComplexExpand[Re[eq[[1]] - eq[[2]]] == 0,
      ComplexExpand[Im[eq[[1]] - eq[[2]]] == 0},
      {eq, complexEqns}]] // ColumnForm, "real eqns :"];
  (* select real eqns linear in {e,f,g,h} *)
  Echo[(linearEqns = Select[realEqns,
    (Total@Exponent[#, {e, f, g, h}] == 4 &)] // ColumnForm, "linear eqns :"];
  Echo[Length[linearEqns], "number of linear eqns (some redundant) :"];
  (* Solve for {e,f,g,h} in terms of {a,b,c,d} *)
  soln = Solve[linearEqns, {e, f, g, h}, Reals][[1]];
  (* Results *)
  Echo[M /. soln // Simplify // MatrixForm, "arb member of SU(2) :"];
  Echo[normRule = (complexEqns[[1]] /. {Equal -> Rule}),
    "norm rule from first complex eqn :"];
  Echo[MatrixForm[Simplify[M /. soln] /. normRule],
    "constrained form for members of SU(2) :"];
  proof[(Simplify[M /. soln] /. normRule) ==  $\begin{pmatrix} a + b i & c + d i \\ -c + d i & a - b i \end{pmatrix}$ ]];
]
```


» complex eqns : $a^2 + b^2 + c^2 + d^2 == 1$
 $a e + b f + c g + d h + i (b e - a f + d g - c h) == 0$
 $a e + b f + c g + d h + i (-b e + a f - d g + c h) == 0$
 $e^2 + f^2 + g^2 + h^2 == 1$
 $-c e - i d e - i c f + d f + a g + i b g + i a h - b h == 1$

» real eqns : $-1 + a^2 + b^2 + c^2 + d^2 == 0$
 $a e + b f + c g + d h == 0$
 $b e - a f + d g - c h == 0$
 $a e + b f + c g + d h == 0$
 $-b e + a f - d g + c h == 0$
 $-1 + e^2 + f^2 + g^2 + h^2 == 0$
 $-1 - c e + d f + a g - b h == 0$
 $-d e - c f + b g + a h == 0$

» linear eqns : $a e + b f + c g + d h == 0$
 $b e - a f + d g - c h == 0$
 $a e + b f + c g + d h == 0$
 $-b e + a f - d g + c h == 0$
 $-1 - c e + d f + a g - b h == 0$
 $-d e - c f + b g + a h == 0$

» number of linear eqns (some redundant) : 6

» arb member of SU(2): $\begin{pmatrix} a + i b & c + i d \\ -\frac{c - i d}{a^2 + b^2 + c^2 + d^2} & \frac{a - i b}{a^2 + b^2 + c^2 + d^2} \end{pmatrix}$

» norm rule from first complex eqn : $a^2 + b^2 + c^2 + d^2 \rightarrow 1$

» constrained form for members of SU(2) : $\begin{pmatrix} a + i b & c + i d \\ -c + i d & a - i b \end{pmatrix}$

QED ■

Unitary 2 × 2 Matrices of Unit Determinant

```
In[31]:= Module[{SU2 =  $\begin{pmatrix} a + b i & c + d i \\ -c + d i & a - b i \end{pmatrix}$ },
  Echo[Det[SU2], "determinant of special form of members of SU(2) :"];
  proof@Echo[(Det[SU2] /. normRule) == 1, "SU(2) matrices are unit radii in  $\mathbb{R}^4$  :"]];
```

» determinant of special form of members of SU(2) : $a^2 + b^2 + c^2 + d^2$

» SU(2) matrices are unit radii in $\mathbb{R}^4$: True

QED ■

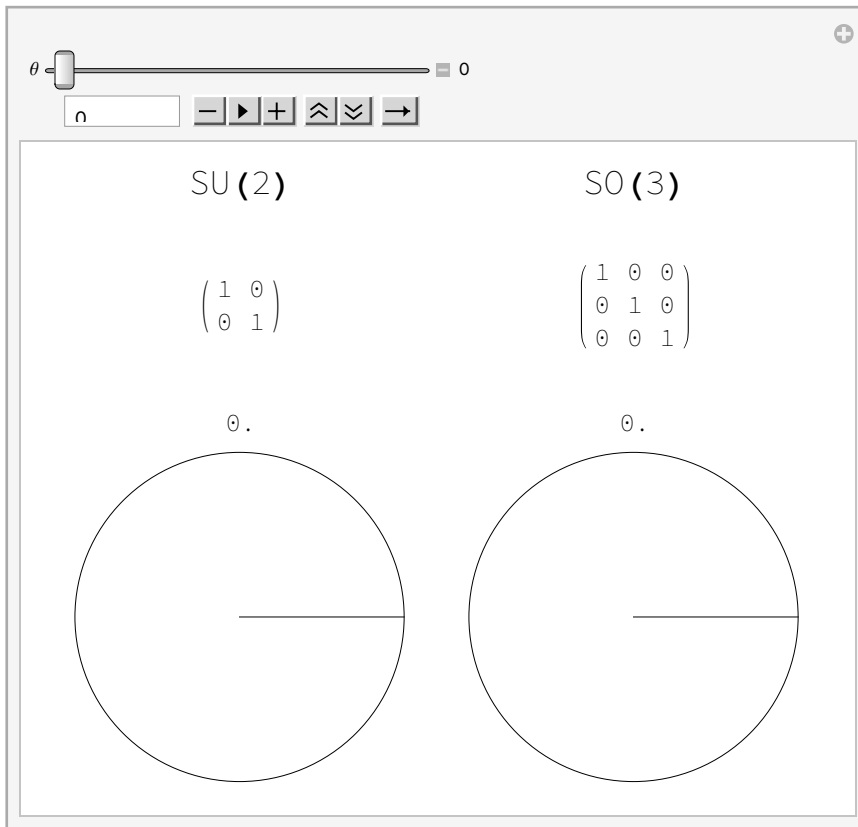
Topological Tearing

Here we can see the “topological tearing” mentioned in the inventories above. The input to the Manipulate is an angle of rotation from 0 to 4π (720°) about the Z axis. The two columns show representations of SU(2) and SO(3) computed straight from the matrix exponentials, plus extractions of the angle from the representations and a “clock-style” visualization of the angle. The demonstration shows that one full rotation of 2π in SO(3) corresponds to half a rotation in SU(2), furnishing intuition about the double covering. Of course, the double covering pertains in exactly the same way to any axis of rotation. Things to try include animating the full range of θ from 0 to 720° and picking various angles to see

the relationships in depth.

```
In[32]:= Manipulate[
  With[{mot = Chop@MatrixExp[θ T[[3]]], moj = Chop@MatrixExp[θ J[[3]]],
    deg = N[°], o = {0, 0}, box = {200, 100}},
  Module[{
    angT = N@Arg[mot[[2, 2]]],
    angJ = N@ArcTan[moj[[1, 1]], moj[[2, 1]]],
    Grid[{
      Style[#, Bold, 18] & /@ {"SU(2)", "SO(3)"},
      {Pane[MatrixForm@mot, box, Alignment → Center],
       Pane[MatrixForm@moj, box, Alignment → Center]},
      {angT / deg, angJ / deg},
      {Graphics[{Circle[o, 1], Line[{o, {Cos[angT], Sin[angT]}]}]},
       Graphics[{Circle[o, 1], Line[{o, {Cos[angJ], Sin[angJ]}]}]} } ]],
  {{θ, 0°}, 0, 720°, 10°, Appearance → {"Open", "Labeled"}}]
```

Out[32]=



Center Inclusion

Proof that I and $(-I)$ commute with generators of both groups.

The **center** of a group is the set of all elements that commute with all other elements. The following shows that I and $-I$ commute with the generators of both $SU(2)$ and $SO(3)$, so that both I and $-I$ are

potentially in the centers of both groups. Below, we show that both I and $-I$ are exactly the center of $SU(2)$, and that I is exactly the center of $SO(3)$. The fact that $-I$ commutes with the generators J of $SO(3)$ is irrelevant because $-I$ is not a group element of $SO(3)$.

```
In[33]:= ClearAll[commM, centerCommutesWithGenerators];
commM[A_, B_] := A.B - B.A;
centerCommutesWithGenerators[g_, dim_] :=
  proof@(And @@ With[{SU2Center = {IdentityMatrix[dim], -IdentityMatrix[dim]},
    zeroMatrix2 = ConstantArray[0, {dim, dim}]}],
    Table[commM[#, g[[k]]] & /@ SU2Center == {zeroMatrix2, zeroMatrix2}, {k, 3}]]];
centerCommutesWithGenerators[T, 2];
centerCommutesWithGenerators[J, 3];
```

QED ■

QED ■

Exclusivity: proof that the only solutions to commutativity are multiples of I

For $SU(2)$

The only solutions to the commutation equations of a general 2×2 matrix are multiples of the identity. This fact follows from Schur's lemma (after proof of irreducibility of the representation), but it's also available via the following, elementary demonstration.

```
In[38]:= Module[{m, vars, sol, center},
  Echo[{m = {{a, b}, {c, d}}}] // MatrixForm, "general 2x2 matrix :";
  vars = {a, b, c, d};
  (* All matrices that commute with SU(2) generators T *)
  Echo[sol = Quiet@Solve[Table[commM[m, T[[k]]] == {{0, 0}, {0, 0}}, {k, 3}], vars][[1]],
    "all matrices that commute with T :"];
  (* m must be a multiple of the identity *)
  Echo[{m /. sol}] // MatrixForm, "general commuting matrix for SU(2) :";
  (* Apply group constraint: Det[m]==1 to show only I and -I obtain *)
  Echo[{center = Quiet@Solve[{Det[m /. sol] == 1}, {a}]}], "center of SU(2) :";
  proof[Length[center] == 2 &&
    (m /. sol /. center) == {-IdentityMatrix[2], IdentityMatrix[2]}]]];
```

» general 2x2 matrix : $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$

» all matrices that commute with T : $\{b \rightarrow 0, c \rightarrow 0, d \rightarrow a\}$

» general commuting matrix for SU(2) : $\begin{pmatrix} a & 0 \\ 0 & a \end{pmatrix}$

» center of SU(2) : $\{\{a \rightarrow -1\}, \{a \rightarrow 1\}\}$

QED ■

The above shows that for $SU(2)$, off-diagonals are forced to 0 and the diagonals are forced to be equal,

leaving only ± 1 once the determinant is constrained.

For SO(3)

Using exactly the same technique as above

```
In[39]:= Module[{m, vars, sol, center, zero3 = ConstantArray[0, {3, 3}]},
  m = Array[x, {3, 3}];
  vars = Flatten[m];
  (* All matrices that commute with SO(3) generators J*)
  sol = Quiet@Solve[Table[commM[m, J[[k]]] == zero3, {k, 3}], vars][[1]];
  (* m must be a multiple of the identity*)
  Echo[m /. sol // MatrixForm, "General commuting matrix for SO(3) :"];
  (* Apply group constraint: Det[m]==1*)
  Echo[{center = Solve[{Det[m /. sol] == 1}, {x[1, 1]}]} // ColumnForm,
    "center of SO(3) :"];
  (* the only real solution is c=1, which is center[[1]]*)
  proof[Length[center] == 3 && (m /. sol /. center[[1]]) == IdentityMatrix[3]]];
```

» General commuting matrix for SO(3) :
$$\begin{pmatrix} x[1, 1] & 0 & 0 \\ 0 & x[1, 1] & 0 \\ 0 & 0 & x[1, 1] \end{pmatrix}$$

» center of SO(3) :
$$\begin{aligned} &\{x[1, 1] \rightarrow 1\} \\ &\{x[1, 1] \rightarrow -(-1)^{1/3}\} \\ &\{x[1, 1] \rightarrow (-1)^{2/3}\} \end{aligned}$$

QED ■

Component Extraction from Trace Inner Product

The objective of this section is first to prove the following **trace formulas** (traces of matrix products) for extracting the components of vectors in (elements of) the Lie algebra $\mathfrak{g}$ in the J and T bases:

$$V_{J_k} = -\frac{1}{2} \text{Tr}(V J_k) \quad \text{--- } k^{\text{th}} \text{ component of } V \text{ in the } J \text{ basis, trace of the matrix product } V J_k \quad (1)$$

- Formally, the inner product of real matrices is $\langle A, B \rangle \equiv \text{Tr}(A^T B)$. In our case, write $\langle J_k, V \rangle = \text{Tr}(J_k^T V) = -\text{Tr}(J_k V) = -\text{Tr}(V J_k)$ because the J_k are anti-symmetric. Finally, normalize because generators J_k , while an orthogonal basis, are not of unit length. The k -th component of V is $\langle J_k, V \rangle / \langle J_k, J_k \rangle = -\frac{1}{2} \text{Tr}(V J_k)$ because $\langle J_k, J_k \rangle = 2$:

```
In[40]:= Table[Tr[J[[k]]^T.J[[k]]], {k, 3}]
Out[40]= {2, 2, 2}
```

$$V_{T_k} = -2 \text{Tr}(V T_k) \quad \text{--- } k^{\text{th}} \text{ component of } V \text{ in the } T \text{ basis, trace of the matrix product } V T_k \quad (2)$$

- Formally, the inner product of complex matrices is $\langle A, B \rangle \equiv \text{Tr}(A^\dagger B)$. In our case, write $\langle T_k, V \rangle = \text{Tr}(T_k^\dagger V) = -\text{Tr}(T_k V) = -\text{Tr}(V T_k)$ because the T_k are anti-Hermitian. Finally, normalize because generators T_k , while an orthogonal basis, are not of unit length. The k -th component of V is $\langle T_k, V \rangle / \langle T_k, T_k \rangle = -2 \text{Tr}(V T_k)$ because $\langle T_k, T_k \rangle = \frac{1}{2}$:

```
In[41]:= Table[Tr[T[[k]]†.T[[k]]], {k, 3}]
```

```
Out[41]= {1/2, 1/2, 1/2}
```

Notational Warnings

- Examine notation meticulously, carefully distinguishing mathematical notation from Wolfram expressions. For example, $\text{Tr}[A, B]$, in Wolfram, is not the trace of the Lie bracket $[A, B]$, it's the "generalized trace," a complication we don't need! Instead, $\text{Tr}[\text{commM}[A, B]]$ is trace of the Lie bracket.
- In a space of complex or real matrices, $\text{Tr}(A^\dagger B)$ is the inner product, where $A^\dagger B$ is the ordinary matrix product of $A^\dagger$ with B . In Wolfram, the ordinary matrix product is denoted by a low dot: $(A^\dagger \cdot B)$.
 - With real matrices, the conjugate transpose $A^\dagger$ is just transpose A^T , so the dagger notation suffices for both real and complex matrices.
- Wolfram function-call is denoted with with square brackets, $f[A, B]$, easy to confuse with Lie brackets $[A, B]$ in non-Wolfram, mathematical notation. In Wolfram, we denote Lie bracket in a matrix representation via a call of the function `commM`, as in `commM[A, B]`, which is defined as $(A \cdot B - B \cdot A)$.

Trace Formula in basis J

```
In[42]:= ClearAll[V, V1, V2, V3, VJ, VT];
```

Begin with the easy identity, checked by calculation:

$$\langle J_i, J_j \rangle = \text{Tr}(J_i^\dagger J_j) = 2 \delta_{ij} \quad (3)$$

```
In[43]:= Table[Tr[J[[i]]†.J[[j]]], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[43]//MatrixForm=
```

$$\begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

- Set up V in the J basis

```
In[44]:= V = {V1, V2, V3};
```

```
(VJ = V.J) // MatrixForm (* expansion of V in J basis *)
```

```
Out[45]//MatrixForm=
```

$$\begin{pmatrix} 0 & -V3 & V2 \\ V3 & 0 & -V1 \\ -V2 & V1 & 0 \end{pmatrix}$$

- Recover VJ 's k -th component via normalized inner product with J_k , multiplying by $\frac{1}{2}$ to cancel $\langle J_k, J_k \rangle$:

```
In[46]:= proof[V == Table[Tr[ $\frac{1}{2}$  VJT.J[k]] // Simplify, {k, 3}]]];
```

QED ■

Trace Formula in basis T

Begin with the easy identity, checked by calculation:

$$\langle T_i, T_j \rangle = \text{Tr}(T_i^\dagger T_j) = \frac{1}{2} \delta_{ij} \quad (4)$$

```
In[47]:= Table[Tr[T[[i]]†.T[[j]]], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[47]//MatrixForm=
```

$$\begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{2} \end{pmatrix}$$

■ Set up V in the T basis

```
In[48]:= V = {V1, V2, V3};
```

```
(VT = V.T) // MatrixForm (* expansion of V in T basis *)
```

```
Out[49]//MatrixForm=
```

$$\begin{pmatrix} -\frac{i V3}{2} & -\frac{i V1}{2} - \frac{V2}{2} \\ -\frac{i V1}{2} + \frac{V2}{2} & \frac{i V3}{2} \end{pmatrix}$$

■ Recover VT 's k -th component via normalized inner product with by T_k , multiplying by 2 to cancel $\langle T_k, T_k \rangle$:

```
In[50]:= proof[V == Table[Tr[2 VT†.T[k]] // ComplexExpand // Simplify, {k, 3}]]];
```

```
ClearAll[V, V1, V2, V3, VJ, VT];
```

QED ■

Final Coordinate-Extraction Functions

```
In[52]:= ClearAll[getJCoeffs, getTCoeffs];
```

```
getJCoeffs[Jx_] := Table[ $\frac{1}{2}$  Tr[JxT.J[k]], {k, 3}];
```

```
getTCoeffs[Tx_] := Table[2 Tr[Tx†.T[k]] // ComplexExpand, {k, 3}];
```

Derivation via the Killing Form

Every element $X \in \mathfrak{g}$ defines a linear map $\text{ad}_X(Z) = [X, Z]$ out of $\mathfrak{g}$'s definitional Lie bracket. Intrinsic geometry of a Lie algebra $\mathfrak{g}$ comes from its **Killing form**, the trace of such maps over two elements X, Y of $\mathfrak{g}$:

$$B(X, Y) = \text{Tr}(\text{ad}_X \circ \text{ad}_Y)$$

The result is a real number.

- While standard, *this is a difficult notation*. ad_X and ad_Y are not operators here, but matrices constructed from expansions of X and Y bracketed on a basis of $\mathfrak{su}(2)$. The following formalizations should clarify.

For *simple* Lie Algebras, like $\mathfrak{so}(3)$ and $\mathfrak{su}(2)$, Schur's Lemma implies that any invariant, symmetric, bilinear form is proportional to the Killing form, so the basis trace formulas above are proportional to the Killing forms. See Georgi, Humphreys, and Fuchs in the References.

- A **simple Lie algebra** is a non-Abelian algebra with no non-trivial **ideals**, that is, no sub-algebras that “trap” the Lie bracket.

```
In[55]:= Module[{
  X, Y, (* arbitrary elements of g, X and Y: *)
  xVec = {x1, x2, x3}, (* coefficients of expansion in basis T *)
  yVec = {y1, y2, y3},
  adX, adY, (* 'ad' matrices (ad-mats) in su(2) *)
  killingB, traceForm},
(* expansions in basis T*)
X = xVec.T;
Y = yVec.T;
(* ad-mats ad_X, ad_Y in basis T via Lie bracket commM *)
(* (ad_X)_{ij} is the component of [X, T_j] along T_i *, ditto Y *)
(* Final Traspose swaps Wolfram rows for our needed columns. *)
Echo[adX = Table[getTCoeffs[commM[X, T[[j]]]], {j, 3}]^T // FullSimplify) //
  MatrixForm, "ad_X = <[X, T_j], T_i> :"];
Echo[adY = Table[getTCoeffs[commM[Y, T[[j]]]], {j, 3}]^T // FullSimplify) //
  MatrixForm, "ad_Y = <[Y, T_j], T_i> :"];
(* Killing form B(X,Y)=Tr[ad_X.ad_Y] *)
killingB = Tr[adX.adY] // Expand;
(* Compute the Trace Form: k Tr[X.Y] *)
(* for su(2) in basis T, k = 4 *)
traceForm = 4 Tr[X.Y] // Expand;
Echo[killingB, "Killing form B(X,Y): "];
Echo[traceForm, "Trace form 4 Tr(X.Y): "];
(* They are identical for all X and Y *)
proof[killingB === traceForm]]];
```

$$\gg \text{ad}_X = \langle [X, T_j], T_i \rangle : \begin{pmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{pmatrix}$$

$$\gg \text{ad}_Y = \langle [Y, T_j], T_i \rangle : \begin{pmatrix} 0 & -y_3 & y_2 \\ y_3 & 0 & -y_1 \\ -y_2 & y_1 & 0 \end{pmatrix}$$

$$\gg \text{Killing form } B(X,Y): -2 x_1 y_1 - 2 x_2 y_2 - 2 x_3 y_3$$

$$\gg \text{Trace form } 4 \text{ Tr}(X.Y): -2 x_1 y_1 - 2 x_2 y_2 - 2 x_3 y_3$$

QED ■

Conversion functions

```
In[56]:= ClearAll[JtoT, TtoJ];
JtoT[Jx_] := Total[getJCoeffs[Jx] * T];
TtoJ[Tx_] := Total[getTCoeffs[Tx] * J];
```

$T \longleftrightarrow J$ Round-Trip Identity

```
In[59]:= ClearAll[testCoeffs, testJ, testT];
testCoeffs = {a, b, c};
Echo[(testJ = Total[testCoeffs * J]) // MatrixForm, "testJ :"];
Echo[(testT = Total[testCoeffs * T]) // MatrixForm, "testT :"];
ClearAll[TJRoundTrip];
TJRoundTrip[] :=
  (Expand[JtoT[TtoJ[testT]]] == testT &&
   Expand[TtoJ[JtoT[testJ]]] == testJ);
proof@TJRoundTrip[];

» testJ: 
$$\begin{pmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{pmatrix}$$

» testT: 
$$\begin{pmatrix} -\frac{ic}{2} & -\frac{ia}{2} - \frac{b}{2} \\ -\frac{ia}{2} + \frac{b}{2} & \frac{ic}{2} \end{pmatrix}$$

QED ■
```

Prove $J \longleftrightarrow T$ Isomorphism

In addition to round-trip identity, we must show that $J \longleftrightarrow T$ preserves the Lie bracket.

Let X and Y be arbitrary elements in the J -span or the T -span.

$JtoT$ preserves the Lie Bracket iff $JtoT([X, Y])$ equals $[JtoT(X), JtoT(Y)]$. Ditto $TtoJ$.

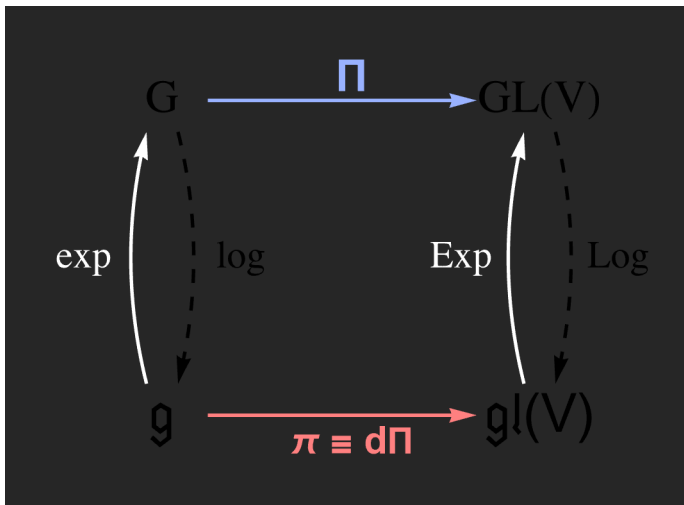
```
In[66]:= ClearAll[X, Y, x1, x2, x3, y1, y2, y3, TJisomorphism];
X[M_] := Total[{x1, x2, x3} * M];
Y[M_] := Total[{y1, y2, y3} * M];
TJisomorphism[T0_, X_, Y_] :=
  Simplify[T0[commM[X, Y]] == commM[T0[X], T0[Y]]];
proof@(
  TJisomorphism[JtoT, X[J], Y[J]] &&
  TJisomorphism[TtoJ, X[T], Y[T]]);
QED ■
```


Explicit Mappings

The following diagram addresses the question “How do I know that exponentiating my generators in $\mathfrak{g}$ will correctly produce physical transformations from my group G on my vectors in V ?”

- G is the abstract Lie group (the manifold of symmetries) and $\mathfrak{g}$ is its Lie algebra (tangent space of generators at the identity of G).
- $GL(V)$ (General Linear Group) is the space of all invertible matrices that act on a physical vector space V (e.g., $V = \mathbb{R}^3$ for vectors or $V = \mathbb{C}^2$ for spinors).
- $\mathfrak{gl}(V)$ (General Linear Algebra) is the space of all matrices acting on V , the “matrix home” for generators.
- Π (Group Representation) is the map that converts an abstract symmetry $g \in G$ into a concrete matrix $M \in GL(V)$.
- $\pi \approx d\Pi$ (Algebra Representation) is the map that converts an abstract generator $X \in \mathfrak{g}$ into a matrix $\pi(X) \in \mathfrak{gl}(V)$. In physics, we call π the “differential” or infinitesimal” version of Π .

Out[71]=



Map the T-basis to the J-basis

T Basis → Matrix Rep

```
In[72]:= proof[2 & T == Table[PauliMatrix[i], {i, 3}]];
```

QED ■

```
In[73]:=  $\pi$ Map[TMat_] :=
Module[{a, b, c},
  {a, b, c} =  $\frac{1}{2}$  Tr[TMat.#] & /@ (2  $\frac{1}{2}$  T);
  {a, b, c}.J]
(* generators-to-generators *)
(MatrixForm@*FullSimplify) /@ ( $\pi$ Map[ $\theta$  #] & /@ T)
```

```
Out[74]= { $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -\theta \\ 0 & \theta & 0 \end{pmatrix}$ ,  $\begin{pmatrix} 0 & 0 & \theta \\ 0 & 0 & 0 \\ -\theta & 0 & 0 \end{pmatrix}$ ,  $\begin{pmatrix} 0 & -\theta & 0 \\ \theta & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$ }
```

Group Representation Map Π

```
With[{ $\sigma$  = PauliMatrix},
   $\Pi$ map[U_] :=
  Table[
     $\frac{1}{2}$  Tr[ $\sigma$ [i].U. $\sigma$ [j].U†],
    {i, 3}, {j, 3}]]];
(* rotations-to-rotations *)
(MatrixForm@*FullSimplify@*ComplexExpand) /@ ( $\Pi$ map[ $\theta$  #] & /@ T)
```

```
Out[76]= { $\begin{pmatrix} \frac{\theta^2}{4} & 0 & 0 \\ 0 & -\frac{\theta^2}{4} & 0 \\ 0 & 0 & -\frac{\theta^2}{4} \end{pmatrix}$ ,  $\begin{pmatrix} -\frac{\theta^2}{4} & 0 & 0 \\ 0 & \frac{\theta^2}{4} & 0 \\ 0 & 0 & -\frac{\theta^2}{4} \end{pmatrix}$ ,  $\begin{pmatrix} -\frac{\theta^2}{4} & 0 & 0 \\ 0 & -\frac{\theta^2}{4} & 0 \\ 0 & 0 & \frac{\theta^2}{4} \end{pmatrix}$ }
```

```
In[77]:= proof@With[{id2 = IdentityMatrix[2],
  id3 = IdentityMatrix[3]},
   $\Pi$ map[-id2] ==  $\Pi$ map[id2] == id3];
```

QED ■

```
In[78]:= ClearAll[commutativeDiagram];
commutativeDiagram[X_] :=
FullSimplify[
  MatrixExp[ $\pi$ Map[X]] ==
   $\Pi$ map[MatrixExp[X]] //
  ComplexExpand
];
proof[And@@ commutativeDiagram /@ ( $\theta$  T)];
```

QED ■

The Big Beautiful Example

Here we show that a 3-vector rotated by an element of $SU(2)$ maps to the same vector rotated by an element of $SO(3)$. The reverse is twoS-to-one, as amply demonstrated above.

$\mathbb{R}^3$ Vector $\longleftrightarrow$ h_σ (Matrix-Vector)

```
In[81]:= ClearAll[toHσ, fromHσ];
toHσ[v_List] := Total[v * T];
fromHσ[mat_] := Table[-2 Tr[mat.T[[k]]], {k, 3}];
```

Rotation Parameters

```
In[84]:= Echo[vR3 = {x, y, z}, "vR3 :"];
{φ, θ, ψ} = {α, β, γ};
» vR3 : {x, y, z}
```

$SO(3)$ Rotation Matrix

$$R = R_z(\phi).R_y(\theta).R_x(\psi) = e^{\phi J_3} e^{\theta J_2} e^{\psi J_1}$$

```
In[86]:= Echo[ (R = MatrixExp[φ J[[3]]].MatrixExp[θ J[[2]]].MatrixExp[ψ J[[1]]]) // MatrixForm,
"Rotation Matrix RR3 :"];
Echo[ (vPrimeR3 = Simplify[R.vR3]) // MatrixForm, "v'R3 = RR3.vR3 :"];
» Rotation Matrix RR3 :

$$\begin{pmatrix} \cos[\alpha] \cos[\beta] & -\cos[\gamma] \sin[\alpha] + \cos[\alpha] \sin[\beta] \sin[\gamma] & \cos[\alpha] \cos[\gamma] \sin[\beta] + \sin[\alpha] \sin[\gamma] \\ \cos[\beta] \sin[\alpha] & \cos[\alpha] \cos[\gamma] + \sin[\alpha] \sin[\beta] \sin[\gamma] & \cos[\gamma] \sin[\alpha] \sin[\beta] - \cos[\alpha] \sin[\gamma] \\ -\sin[\beta] & \cos[\beta] \sin[\gamma] & \cos[\beta] \cos[\gamma] \end{pmatrix}$$

» v'R3 = RR3.vR3 :

$$\begin{pmatrix} \sin[\alpha] (-y \cos[\gamma] + z \sin[\gamma]) + \cos[\alpha] (x \cos[\beta] + \sin[\beta] (z \cos[\gamma] + y \sin[\gamma])) \\ \cos[\alpha] (y \cos[\gamma] - z \sin[\gamma]) + \sin[\alpha] (x \cos[\beta] + \sin[\beta] (z \cos[\gamma] + y \sin[\gamma])) \\ -x \sin[\beta] + \cos[\beta] (z \cos[\gamma] + y \sin[\gamma]) \end{pmatrix}$$

```

$SU(2)$ Unit Quaternion

$$Q_{\phi\theta\psi} = e^{\phi T_3} e^{\theta T_2} e^{\psi T_1}$$

```
In[88]:= Echo[
(Q = MatrixExp[φ T[[3]]].MatrixExp[θ T[[2]]].MatrixExp[ψ T[[1]]]) // MatrixForm, "Qφθψ :"];
» Qφθψ :

$$\begin{pmatrix} e^{-\frac{i\alpha}{2}} \cos[\frac{\beta}{2}] \cos[\frac{\gamma}{2}] + i e^{-\frac{i\alpha}{2}} \sin[\frac{\beta}{2}] \sin[\frac{\gamma}{2}] & -e^{-\frac{i\alpha}{2}} \cos[\frac{\gamma}{2}] \sin[\frac{\beta}{2}] - i e^{-\frac{i\alpha}{2}} \cos[\frac{\beta}{2}] \sin[\frac{\gamma}{2}] \\ e^{\frac{i\alpha}{2}} \cos[\frac{\gamma}{2}] \sin[\frac{\beta}{2}] - i e^{\frac{i\alpha}{2}} \cos[\frac{\beta}{2}] \sin[\frac{\gamma}{2}] & e^{\frac{i\alpha}{2}} \cos[\frac{\beta}{2}] \cos[\frac{\gamma}{2}] - i e^{\frac{i\alpha}{2}} \sin[\frac{\beta}{2}] \sin[\frac{\gamma}{2}] \end{pmatrix}$$

```

Adjoint Action in $\mathfrak{h}_\sigma$

$$v_{h_0} = \text{toH}_\sigma(v_{\mathbb{R}^3})$$

```
In[89]:= Echo[(vHσ = toHσ[vR3]) // MatrixForm, "vhσ :"];
```

$$\gg v_{h_\sigma} : \begin{pmatrix} -\frac{i z}{2} & -\frac{i x}{2} - \frac{y}{2} \\ -\frac{i x}{2} + \frac{y}{2} & \frac{i z}{2} \end{pmatrix}$$

$$Q_{\phi\theta\psi} v_{h_\sigma} Q_{\phi\theta\psi}^\dagger$$

```
In[90]:= Echo[(vPrimeHσ = Simplify[ComplexExpand[Q.vHσ.Q†]]) // MatrixForm,
```

```
"v'_{hσ}=Q_{φθψ} v_{hσ} Q_{φθψ}^† :"];
```

$$\gg v'_{h_\sigma} = Q_{\phi\theta\psi} v_{h_\sigma} Q_{\phi\theta\psi}^\dagger : \begin{pmatrix} -\frac{1}{2} i (-x \sin[\beta] + \cos[\beta] (z \cos[\gamma] + y \sin[\gamma])) & \\ \frac{1}{2} (-i \cos[\alpha] + \sin[\alpha]) (x \cos[\beta] + \cos[\gamma] (i y + z \sin[\beta]) + (-i z + y \sin[\beta]) \sin[\gamma] \end{pmatrix}$$

Proofs

Proof 1: Consistency of the Pauli Map

```
In[91]:= Echo[proof[Simplify[
  fromHσ[toHσ[vR3]] == vR3
]], "Pauli Map Round Trip:  $\mathbb{R}^3 \rightarrow \mathfrak{h}_\sigma \rightarrow \mathbb{R}^3$  :"];
```

QED ■

» Pauli Map Round Trip: $\mathbb{R}^3 \rightarrow \mathfrak{h}_\sigma \rightarrow \mathbb{R}^3$: True

Proof 2: $SO(3)$ from $SU(2)$

Matrix elements of $R_{\phi\theta\psi}$ are $R_{ij} = -2 \text{Tr}(T_j Q_{\phi\theta\psi} T_i Q_{\phi\theta\psi}^\dagger)$

```
In[92]:= Echo[(RFromQ = Table[
  -2 Tr[T[[i]].Q.T[[j]].Q†] //
  ComplexExpand // Simplify,
  {i, 3}, {j, 3}]) // MatrixForm, "Rij = -2Tr[Ti·Q·Tj·Q†] :"];
proof[RFromQ == R];
```

» $R_{ij} = -2 \text{Tr}[T_i \cdot Q \cdot T_j \cdot Q^\dagger]$:

$$\begin{pmatrix} \cos[\alpha] \cos[\beta] & -\cos[\gamma] \sin[\alpha] + \cos[\alpha] \sin[\beta] \sin[\gamma] & \cos[\alpha] \cos[\gamma] \sin[\beta] + \sin[\alpha] \sin[\gamma] \\ \cos[\beta] \sin[\alpha] & \cos[\alpha] \cos[\gamma] + \sin[\alpha] \sin[\beta] \sin[\gamma] & \cos[\gamma] \sin[\alpha] \sin[\beta] - \cos[\alpha] \sin[\gamma] \\ -\sin[\beta] & \cos[\beta] \sin[\gamma] & \cos[\beta] \cos[\gamma] \end{pmatrix}$$

QED ■

Proof 3: Equivalence of the Rotated Vectors

Does the $SU(2)$ rotated v'_{h_σ} mat-vec map back to the $SO(3)$ rotated vector $v'_{\mathbb{R}^3}$?

```
In[94]:= proof[Simplify[
  fromHσ[vPrimeHσ] == vPrimeR3
]];
QED ■
```

References

1. Beckman, Brian (2026). *SO(3) Gauge Theory for Classical Mechanics in $\mathcal{UD}$* . Wolfram Community post.
2. Beckman, Brian. (2026) *SU(2) Gauge Theory for Classical Mechanics in $\mathcal{UD}$* . Wolfram Community post.
3. Hall, B. C. (2015). *Lie Groups, Lie Algebras, and Representations: an Elementary Introduction*, 2nd Edition, Springer. (Theorem 3.29)
4. Knapp, A. W. (2002). *Lie Groups Beyond an Introduction*. (Theorem 4.43).
5. Milnor, J. (1976). *Curvatures of Left Invariant Metrics on Lie Groups*. (Adv. Math. 21).
6. Georgi, H. (1999). *Lie Algebras in Particle Physics: From Isospin to Unified Theories*. Westview Press.
7. Humphreys, J. E. (1972). *Introduction to Lie Algebras and Representation Theory*. Springer.
8. Fuchs, J., & Schweigert, C. (1997). *Symmetries, Lie Algebras and Representations: A Graduate Course for Physicists*. Cambridge University Press.